Module 3 — CSS3

> **Objectif** : maîtriser la mise en forme des pages web : sélecteurs, modèle de boîte, mise en page moderne (Flexbox, Grid), responsive design, animations.

1. Qu'est-ce que CSS ?

CSS = *Cascading Style Sheets* (feuilles de styles en cascade). Sert à **présenter** le contenu HTML : couleurs, polices, espacements, positionnement, animations.

	CSS
Type	Langage de feuille de style
Créé en	1996 (Håkon Wium Lie)
Version actuelle	CSS3 (modules continus)
Extension	`.css`

Pourquoi séparer HTML et CSS ?

- **Maintenabilité** : changer la couleur d'un site = modifier une seule ligne.
- **Performance** : le CSS est mis en cache, téléchargé une seule fois pour tout le site.
- **Accessibilité** : le contenu reste lisible sans CSS.

2. Anatomie d'une règle CSS

```
sélecteur {
  propriété : valeur ;
  propriété : valeur ;
}
```

```css
h1 {
  color: #2c3e50;
  font-size: 2rem;
  margin-bottom: 1rem;
}
```

- **Sélecteur** : à qui s'applique la règle (`h1` , `.bouton` , `#menu`).
- **Déclaration** : `propriété: valeur;`
- **Bloc** : entre `{ }` , contient une ou plusieurs déclarations.

3. Comment intégrer CSS au HTML

3.1 CSS externe (recommandé)

`style.css` :

```css
body { background: #f4f4f4; }
```

Dans `index.html` :

```html
<link rel="stylesheet" href="style.css">
```

3.2 CSS interne (`<style>` dans `<head>`)

```html
<head>
  <style>
    body { background: #f4f4f4; }
  </style>
</head>
```

À réserver aux pages uniques (e-mail HTML, page de démo).

3.3 CSS inline (`style="…"`)

```html
<p style="color: red;">Texte rouge</p>
```

⚠️ À **éviter** : non maintenable, mauvaise séparation des responsabilités. Acceptable seulement pour du style dynamique généré en JS.

4. Les sélecteurs

Sélecteurs simples

Sélecteur	Cible
`*`	Tous les éléments
`p`	Toutes les balises `<p>`
`.intro`	Tous les éléments ayant `class="intro"`
`#menu`	L'élément ayant `id="menu"` (unique)
`[type="text"]`	Tous les éléments ayant l'attribut `type="text"`

Combinaisons

Sélecteur	Cible
`h1, h2, h3`	Sélection multiple (h1 OU h2 OU h3)
`article p`	Tous les `<p>` à l'intérieur d'un `<article>` (descendants)
`article > p`	Tous les `<p>` enfants directs de `<article>`
`h2 + p`	Le `<p>` qui suit immédiatement un `<h2>`
`h2 ~ p`	Tous les `<p>` qui suivent un `<h2>` (au même niveau)
`a.externe`	Les `<a>` ayant la classe `externe`

Pseudo-classes

État ou position d'un élément.

```css
a:hover         { color: red; }            /* Au survol */
a:focus         { outline: 2px solid blue; } /* Quand l'élément a le focus clavier */
a:visited       { color: purple; }         /* Liens déjà visités */
button:disabled { opacity: 0.5; }
input:checked   { /* Case cochée */ }
input:invalid   { border-color: red; }

li:first-child  { font-weight: bold; }     /* Premier enfant */
li:last-child   { color: gray; }
li:nth-child(2) { /* Deuxième enfant */ }
li:nth-child(odd)  { background: #eee; }   /* Lignes impaires */
li:nth-child(even) { background: #fff; }
li:nth-of-type(3n+1) { /* Tous les 3 à partir du 1er */ }

p:not(.intro)   { /* Tous les <p> sauf ceux ayant .intro */ }
```

Pseudo-éléments

Permettent de styler une partie d'un élément, ou d'ajouter du contenu.

```css
p::first-line   { font-weight: bold; }
p::first-letter { font-size: 2em; }

p::before { content: "→ "; color: red; }
p::after  { content: " ✓"; }

::selection { background: yellow; }       /* Texte sélectionné à la souris */
```

⚠️ Les pseudo-éléments s'écrivent avec **deux points** `::`. Les pseudo-classes avec **un seul** `:`.

5. La cascade et la spécificité

Quand plusieurs règles ciblent le même élément, laquelle gagne ?

Ordre de résolution

1. `!important` (à éviter absolument).
2. **Spécificité** du sélecteur (voir tableau).
3. **Ordre d'apparition** : la dernière règle écrite gagne en cas d'égalité.

Calcul de spécificité

Type	Poids
Sélecteur inline (`style="…"`)	1000
`#id`	100
`.classe` , `[attribut]` , `:hover`	10
`balise` , `::pseudo-élément`	1
`*`	0

Exemple :

```
p              { color: blue; }    /* Spécificité : 1 */
.intro         { color: green; }   /* Spécificité : 10 */
#unique        { color: red; }     /* Spécificité : 100 */
article p.intro { color: orange; } /* Spécificité : 1 + 1 + 10 = 12 */
```

💡 **Règle d'or** : préférez des **classes** (`.bouton`) plutôt que des `#id` pour styler. Les IDs sont trop spécifiques et difficiles à surcharger.

6. L'héritage

Certaines propriétés sont **héritées** par les descendants (`color` , `font-family` , `font-size` , `line-height` …). D'autres non (`margin` , `padding` , `border` , `background` …).

```
body { color: navy; font-family: sans-serif; }
/* Tous les éléments du body héritent de la couleur et de la police. */
```

Forcer l'héritage : `propriété: inherit;`
Forcer la valeur par défaut : `propriété: initial;`
Annuler : `propriété: unset;`

7. Les unités

Unités absolues

Unité	Sens
`px`	Pixel — la plus utilisée
`pt`	Point (1pt ≈ 1.33px) — pour l'impression
`cm` , `mm` , `in`	Centimètre, millimètre, pouce

Unités relatives

Unité	Relative à
`%`	Le parent
`em`	La taille de police du **parent**
`rem`	La taille de police de `<html>` (recommandée pour les polices)
`vw` / `vh`	1 % de la largeur / hauteur du *viewport*
`vmin` / `vmax`	Plus petit/grand des deux
`ch`	Largeur du caractère "0"

💡 **Bonne pratique** : `html { font-size: 100%; }` (= 16px par défaut), puis utiliser `rem` partout pour les tailles. Ainsi, l'utilisateur peut zoomer.

8. Les couleurs

```
.exemple {
  color: red;                       /* Mot-clé */
  color: #ff0000;                   /* Hexadécimal */
  color: #f00;                      /* Hexa raccourci */
  color: rgb(255, 0, 0);            /* RGB */
  color: rgba(255, 0, 0, 0.5);      /* RGB avec transparence (alpha) */
  color: hsl(0, 100%, 50%);         /* Teinte, Saturation, Luminosité */
  color: hsla(0, 100%, 50%, 0.5);
}
```

Modèle	Quand l'utiliser
HEX	Couleur précise (charte graphique)
RGB/RGBA	Quand on veut de la transparence
HSL/HSLA	Pour créer facilement des variantes (changer juste la luminosité)

9. Les polices (typographie)

```
body {
  font-family: "Inter", "Helvetica Neue", Arial, sans-serif;
  font-size: 1rem;          /* 16px */
  font-weight: 400;         /* 100 (fin) à 900 (gras extrême) */
  font-style: italic;
  line-height: 1.6;         /* Sans unité = multiple de font-size */
  letter-spacing: 0.05em;
  text-align: justify;      /* left | right | center | justify */
  text-transform: uppercase; /* lowercase | capitalize */
  text-decoration: underline; /* none | line-through | overline */
}
```

Polices web (Google Fonts)

```
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;700&display=swap" rel="styl
```

```
body { font-family: "Inter", sans-serif; }
```

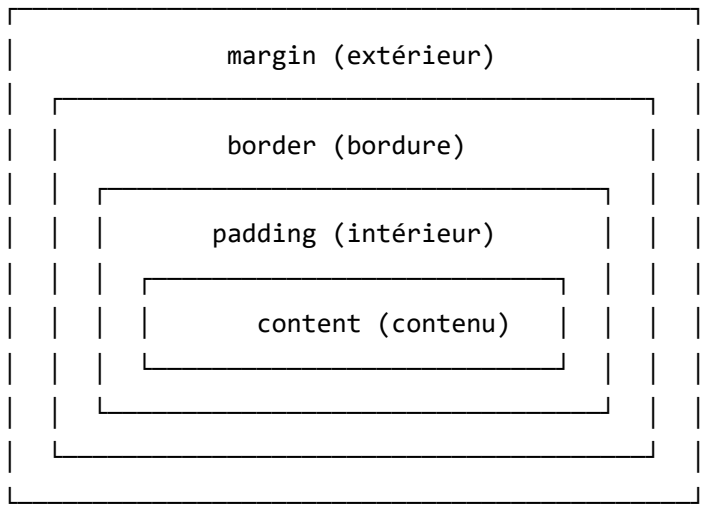
Bonne pratique : pile de polices (fallback)

```
font-family: "Inter", -apple-system, BlinkMacSystemFont,
             "Segoe UI", Roboto, sans-serif;
```

Si la première n'est pas disponible, le navigateur essaie la suivante.

10. Le modèle de boîte (*Box Model*)

Chaque élément HTML est une **boîte rectangulaire** composée de 4 zones.




```css
.boite {
  width: 300px;
  height: 200px;
  padding: 20px;       /* Espace intérieur */
  border: 2px solid #333;
  margin: 16px;        /* Espace extérieur */
}
```

box-sizing

Par défaut (`content-box`), `width` désigne uniquement le contenu, et `padding + border` s'**ajoutent** à la largeur totale. Très contre-intuitif.

```css
*, *::before, *::after {
  box-sizing: border-box;   /* Reset universel recommandé */
}
```

Désormais, `width: 300px` inclut padding et bordure.

Raccourcis margin / padding

```
margin: 10px;                 /* Tous les côtés */
margin: 10px 20px;            /* Haut/bas 10px, gauche/droite 20px */
margin: 10px 20px 30px;       /* Haut 10, droite/gauche 20, bas 30 */
margin: 10px 20px 30px 40px;  /* Haut, droite, bas, gauche (sens horaire) */

margin: 0 auto;               /* Centrage horizontal (avec width fixe) */
```

11. `display`

Définit comment un élément se comporte dans la mise en page.

Valeur	Comportement
`block`	Prend toute la largeur, saute à la ligne (div, p, h1, section…)
`inline`	Suit le flux de texte, pas de largeur/hauteur (span, a, strong…)
`inline-block`	Comme inline mais accepte width/height
`flex`	Active Flexbox (voir section 13)
`grid`	Active CSS Grid (voir section 14)
`none`	Élément **caché** et retiré du flux

`visibility: hidden` cache l'élément mais **garde** son espace ; `display: none` le retire complètement.

12. `position`

```
.element { position: static; }    /* Par défaut, suit le flux */
.element { position: relative; }  /* Reste dans le flux, peut être décalé */
.element { position: absolute; }  /* Sort du flux, se positionne / au parent positionné */
.element { position: fixed; }     /* Fixe / au viewport (reste visible au scroll) */
.element { position: sticky; }    /* Devient fixed à partir d'un seuil de scroll */

.element {
  top: 10px;
  right: 0;
  bottom: 20px;
  left: 50%;
  z-index: 10;   /* Ordre d'empilement (plus c'est grand, plus c'est devant) */
}
```

Exemple : header fixé en haut

```
header {
  position: sticky;
  top: 0;
  background: white;
  z-index: 100;
}
```

13. Flexbox — disposition en 1 dimension

Flexbox aligne facilement des éléments **sur une ligne ou une colonne**.

Activer Flexbox

```
.conteneur {
  display: flex;
}
```

Le conteneur devient *flex container* ; ses enfants directs deviennent *flex items*.

Propriétés du conteneur

```css
.conteneur {
  display: flex;
  flex-direction: row;        /* row | row-reverse | column | column-reverse */
  flex-wrap: wrap;            /* nowrap (défaut) | wrap | wrap-reverse */
  gap: 1rem;                  /* Espace entre les items */

  justify-content: center;    /* Alignement sur l'axe principal */
  /* flex-start | flex-end | center | space-between | space-around | space-evenly */

  align-items: center;        /* Alignement sur l'axe secondaire */
  /* stretch (défaut) | flex-start | flex-end | center | baseline */

  align-content: space-between; /* Pour le multi-ligne */
}
```

Propriétés des items

```css
.item {
  flex-grow: 1;       /* Capacité à grandir (proportionnel) */
  flex-shrink: 1;     /* Capacité à rétrécir */
  flex-basis: 200px;  /* Taille de base avant grow/shrink */

  flex: 1;            /* Raccourci : flex-grow:1; flex-shrink:1; flex-basis:0; */
  flex: 0 0 200px;    /* Taille fixe 200px, ni grandit ni rétrécit */

  align-self: flex-end; /* Surcharge align-items pour cet item */
  order: -1;            /* Réordonner les items visuellement */
}
```

Exemples typiques

Centrer un élément :

```css
.conteneur {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

Barre de navigation :

```css
nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem;
}
```

Grille de cartes responsive :

```css
.grille {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}
.carte {
  flex: 1 1 250px;   /* Min 250px, grandit si possible */
}
```

14. CSS Grid — disposition en 2 dimensions

Grid permet de créer de vraies grilles (lignes ET colonnes).

```css
.conteneur {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;       /* 3 colonnes : 25% / 50% / 25% */
  grid-template-rows: 100px auto 50px;       /* 3 lignes */
  gap: 1rem;                                 /* Espace entre cellules */
}
```

Unités Grid

Unité	Sens
`1fr` , `2fr`	Fraction de l'espace disponible
`auto`	Taille selon le contenu
`repeat(3, 1fr)`	3 colonnes égales
`repeat(auto-fit, minmax(200px, 1fr))`	Autant de colonnes que possible de min. 200px

Unité	Sens
`minmax(100px, 1fr)`	Entre 100px et 1fr

Placement des items

```
.item {
  grid-column: 1 / 3;        /* De la ligne 1 à 3 (sur 2 colonnes) */
  grid-column: span 2;        /* Sur 2 colonnes à partir de la position actuelle */
  grid-row: 2 / 4;
}
```

Zones nommées

```
.layout {
  display: grid;
  grid-template-areas:
    "header header header"
    "nav    main   aside"
    "footer footer footer";
  grid-template-columns: 200px 1fr 200px;
  grid-template-rows: 80px 1fr 50px;
  min-height: 100vh;
}

header { grid-area: header; }
nav    { grid-area: nav; }
main   { grid-area: main; }
aside  { grid-area: aside; }
footer { grid-area: footer; }
```

Grille responsive sans media query

```
.galerie {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 1rem;
}
```

	Flexbox	Grid
Dimension	1D (ligne OU colonne)	2D (lignes ET colonnes)

	Flexbox	Grid
Idéal pour	Navigation, alignement, distribution	Maquette complète, galerie
Contenu vs container	Pilote par le contenu	Pilote par la grille

15. Variables CSS (custom properties)

Permettent de **centraliser** les valeurs (couleurs, espacements, polices…).

```
:root {
  --couleur-primaire: #2c3e50;
  --couleur-accent: #e74c3c;
  --espacement: 1rem;
  --rayon: 8px;
  --police: "Inter", sans-serif;
}

body {
  font-family: var(--police);
  color: var(--couleur-primaire);
}

.bouton {
  background: var(--couleur-accent);
  padding: var(--espacement);
  border-radius: var(--rayon);
}
```

Les variables sont **dynamiques** : on peut les modifier en JS pour créer un thème clair/sombre.

16. Responsive design

Un site **responsive** s'adapte à tous les écrans (mobile, tablette, desktop, TV).

Le viewport

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Cette balise dans le `<head>` est **indispensable** pour que la page soit correctement affichée sur mobile.

Media queries

```
/* Style mobile par défaut (mobile-first) */
.conteneur { padding: 1rem; }

/* Tablette et plus */
@media (min-width: 768px) {
  .conteneur { padding: 2rem; }
}

/* Desktop */
@media (min-width: 1024px) {
  .conteneur {
    padding: 3rem;
    max-width: 1200px;
    margin: 0 auto;
  }
}

/* Impression */
@media print {
  nav, footer { display: none; }
  body { color: black; background: white; }
}

/* Préférence utilisateur */
@media (prefers-color-scheme: dark) {
  body { background: #111; color: #eee; }
}

@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition: none !important; }
}
```

Points de rupture (*breakpoints*) usuels

Appareil	Largeur typique
Mobile	< 600 px
Tablette	600 – 1024 px

Appareil	Largeur typique
Desktop	1024 – 1440 px
Grand écran	> 1440 px

💡 **Approche mobile-first** : écrivez d'abord le CSS pour mobile, puis ajoutez des `min-width` pour les écrans plus grands. C'est plus simple à maintenir.

Images responsives en CSS

```
img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

17. Transitions

Animer le changement d'une propriété de façon fluide.

```
.bouton {
  background: blue;
  transition: background 0.3s ease, transform 0.3s ease;
}

.bouton:hover {
  background: red;
  transform: scale(1.1);
}
```

Syntaxe : `transition: propriété durée fonction délai;`

Fonction (`timing-function`)	Effet
`linear`	Vitesse constante
`ease` (défaut)	Lent au début et à la fin
`ease-in`	Lent au début
`ease-out`	Lent à la fin

Fonction (`timing-function`)	Effet
`cubic-bezier(0.25, 0.1, 0.25, 1)`	Courbe personnalisée

18. Animations (`@keyframes`)

Plus puissantes que les transitions : permettent de définir plusieurs étapes.

```
@keyframes apparition {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}

.carte {
  animation: apparition 0.6s ease-out;
}
```

Avec étapes intermédiaires :

```
@keyframes rebond {
  0%   { transform: translateY(0); }
  50%  { transform: translateY(-20px); }
  100% { transform: translateY(0); }
}

.balle {
  animation: rebond 1s infinite;
}
```

Propriétés d'animation

```css
.element {
  animation-name: apparition;
  animation-duration: 0.6s;
  animation-timing-function: ease;
  animation-delay: 0.2s;
  animation-iteration-count: 1;    /* infinite pour boucle */
  animation-direction: normal;     /* alternate pour aller-retour */
  animation-fill-mode: forwards;   /* Garder l'état final */
}

/* Raccourci */
.element {
  animation: apparition 0.6s ease 0.2s 1 normal forwards;
}
```

19. Transformations 2D et 3D

```css
.element {
  transform: translateX(20px);       /* Déplacement horizontal */
  transform: translate(10px, 20px);  /* X et Y */
  transform: scale(1.5);              /* Agrandissement */
  transform: rotate(45deg);
  transform: skew(10deg, 5deg);

  /* Cumul */
  transform: translate(50px, 0) rotate(45deg) scale(1.2);

  /* 3D */
  transform: rotateY(180deg);
  perspective: 1000px;
}
```

20. Effets visuels

```css
.element {
  border-radius: 8px;                          /* Coins arrondis */
  border-radius: 50%;                          /* Cercle */

  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);    /* Ombre portée */
  text-shadow: 1px 1px 2px black;              /* Ombre de texte */

  background: linear-gradient(to right, #ff7e5f, #feb47b);  /* Dégradé */
  background: radial-gradient(circle, #ff7e5f, #feb47b);

  opacity: 0.8;                                /* Transparence */
  backdrop-filter: blur(10px);                 /* Flou de l'arrière-plan */
  filter: blur(5px) grayscale(50%);            /* Filtres sur l'élément */
}
```

21. Reset CSS

Les navigateurs appliquent des styles par défaut différents. Pour partir d'une base propre :

```css
/* Reset minimaliste recommandé */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html { font-size: 100%; }   /* 16px par défaut */
body { line-height: 1.6; font-family: sans-serif; }

img { max-width: 100%; display: block; }

a { color: inherit; text-decoration: none; }

ul, ol { list-style: none; }

button, input, textarea, select { font: inherit; }
```

22. Méthodologies de nommage : BEM

BEM = *Block, Element, Modifier*. Convention pour nommer les classes CSS.

```
<article class="carte carte--mise-en-avant">
  <img class="carte__image" src="…" alt="">
  <h3 class="carte__titre">Titre</h3>
  <p class="carte__description">…</p>
  <a class="carte__lien carte__lien--externe" href="#">En savoir plus</a>
</article>
```

- `carte` = bloc
- `carte__titre` = élément du bloc
- `carte--mise-en-avant` = variante (modificateur)

Avantages : code lisible, classes plates (faible spécificité), réutilisable.

23. Outils du quotidien

Outil	Usage
DevTools → Éléments	Voir et modifier le CSS d'une page en direct
Live Server (VS Code)	Recharger le navigateur à chaque sauvegarde
Prettier	Formater automatiquement le CSS
Stylelint	Détecter les erreurs et mauvaises pratiques
CSS Stats	Analyser la taille / complexité d'une feuille de style
Coolors	Générer des palettes de couleurs
Google Fonts	Polices web gratuites

24. Quiz d'auto-évaluation

1. Quelle est la différence entre `em` et `rem` ?
2. Quel est le sélecteur le plus spécifique : `.bouton` , `#menu` , `div p` ?
3. Que fait `box-sizing: border-box;` ?

4. Citez la différence entre Flexbox et Grid.
5. Quelle balise `<meta>` est indispensable pour le responsive ?
6. Différence entre `display: none` et `visibility: hidden` ?
7. Quel est le rôle de `@keyframes` ?
8. Pourquoi utiliser des variables CSS ?

> **Réponses** : 1. `em` = parent, `rem` = racine `<html>` · 2. `#menu` (spécificité 100) · 3. Inclut padding/border dans width · 4. Flexbox = 1D, Grid = 2D · 5. `<meta name="viewport">` · 6. `none` retire l'espace, `hidden` le garde · 7. Définir les étapes d'une animation · 8. Centraliser, réutiliser, modifier dynamiquement en JS.

Ressources

- 📚 MDN — Apprendre CSS
- 📚 CSS-Tricks
- 📚 Flexbox Froggy — jeu pour apprendre Flexbox
- 📚 Grid Garden — jeu pour apprendre Grid
- 📚 A Complete Guide to Flexbox
- 📚 A Complete Guide to Grid
- 🎨 Coolors — palettes de couleurs

➡️ **Module suivant** : 04 — JavaScript